Instituto Tecnológico y de Estudios Superiores de Monterrey

# Maestría en Inteligencia Artificial:

## *Spin Voyager & Spin Compass:Avance 4. Modelos alternativos*


**Tania Alicia Caballero Saavedra - A01795957**

**Oscar Enrique García García - A01016093**

**Dante Rosas Fragoso – A01795850**

**Profesor Titular:** Dra. Grettel Barceló Alonso

**Asesor:** Dr. Horacio Martínez Alfaro


Proyecto Integrador

Febrero, 2026

# Índice


1. Introducción . . . . . 1
2. Análisis de Gemini 2.0 Flash-Lite como Modelo de Continuidad . . . . . 2
   2.1. Justificación de la Eficiencia Operativa y de Costos . . . . . 2
   2.2. Rendimiento en Latencia y Ventana de Contexto . . . . . 2
3. Restricciones Técnicas en Google Cloud Platform y su Impacto en el RAG . . . . . 4
   3.1. Gestión de Infraestructura y Límites de Carga . . . . . 4
   3.2. Privacidad y Seguridad de la Información (PII) . . . . . 5
4. Rediseño de Patrones de Limpieza de Documentos (NoiseReducer 2.0) . . . . . 6
   4.1. Tratamiento de Tablas y Datos Estructurados en PDFs . . . . . 7
   4.2. Gestión de Stopwords y Normalización de Dominio . . . . . 8
5. Análisis de Impacto en las Métricas de Desempeño . . . . . 9
   5.1. Mejora Proyectada en el Hit Rate y MRR . . . . . 9
   5.2. Evaluación de la Fidelidad (Faithfulness) y RCC . . . . . 9
6. Integridad de la Información y Gestión de Alucinaciones . . . . . 11
   6.1. La «Verdad Documental» en Finanzas . . . . . 11
   6.2. Model-Assisted Generation (MAG) vs. RAG Tradicional . . . . . 11
7. Consideraciones Operativas y Experiencia del Spinner . . . . . 12
   7.1. Integración en Slack y Tono de Comunicación . . . . . 12
   7.2. Costos y Sostenibilidad a Largo Plazo . . . . . 13
8. Resultados . . . . . 14
   8.1. Entorno Experimental y Descripción del Notebook de Calibración . . . . . 14
      8.1.1. Arquitectura General del Notebook . . . . . 14
      8.1.2. Inicialización del entorno y configuración global . . . . . 15
      8.1.3. Ingesta y Preprocesamiento Documental . . . . . 15
      8.1.4. Implementación del Algoritmo Max-Min Semantic Chunking . . . . . 16
      8.1.5. Implementación del Pipeline RAG y Evaluación del Desempeño . . . . . 17
      8.1.6. Generación del Golden Dataset con LLM (Gemini) . . . . . 19

8.1.7. Motor de Evaluación y Reporte de Métricas RAG . . . . . 20
9. Conclusiones y Recomendaciones para la Implementación . . . . . 21
10. Referencias . . . . . 23

# 1. Introducción

La implementación de **Spin Compass** ha alcanzado una fase crítica de refinamiento técnico, transitando desde una arquitectura *baseline* validada hacia la exploración de modelos alternativos que optimicen la relación entre costo, latencia y precisión. El éxito inicial, fundamentado en la metodología de recuperación aumentada por generación (**RAG**), ha demostrado que la integridad de la información es el desafío primordial en el ecosistema documental de Spin. A diferencia de los sistemas de inteligencia artificial tradicionales, donde la métrica dominante suele ser la exactitud predictiva, **Spin Compass** debe garantizar que el «*fragmento de verdad*» o **Gold Chunk** sea recuperado de manera íntegra para evitar alucinaciones que podrían comprometer las decisiones financieras de los colaboradores o Spinners. El paso hacia el Avance 4 implica no solo una actualización del modelo generativo hacia **Gemini 2.0 Flash-Lite**, sino también una reestructuración profunda de los patrones de limpieza de documentos para alinearse con las restricciones técnicas inherentes a **Google Cloud Platform (GCP)**.

La arquitectura *baseline* estableció que el algoritmo **Naive RAG**, apoyado en estrategias de segmentación semántica como **Max-Min Semantic Chunking**, ofrece resultados superiores en comparación con métodos de ventana deslizante o **Growing Window**. No obstante, la experiencia operativa sugiere que la calidad de la respuesta final está intrínsecamente ligada a la pureza de los datos de entrada. La presencia de ruido estructural en los documentos oficiales, tales como encabezados, pies de página, avisos legales y metadatos técnicos, distorsiona la representación vectorial en **AlloyDB**, provocando fallos de recuperación que el modelo generativo no puede corregir por sí solo. Por tanto, esta etapa de investigación se centra en la modificación quirúrgica de los patrones de limpieza, justificando cada cambio desde la perspectiva de la eficiencia de *tokens* y la fidelidad semántica necesaria para un entorno corporativo de alta exigencia.

# 2. Análisis de Gemini 2.0 Flash-Lite como Modelo de Continuidad

## 2.1. Justificación de la Eficiencia Operativa y de Costos

La selección de **Gemini 2.0 Flash-Lite** como el motor de continuidad para **Spin Compass** responde a una necesidad estratégica de escalabilidad y eficiencia económica. En el marco de **Google Cloud Platform**, los costos de inferencia y procesamiento de *tokens* representan una parte significativa del presupuesto operativo, estimado inicialmente en USD 600 mensuales para la infraestructura completa. **Gemini 2.0 Flash-Lite** se posiciona como el modelo más *cost-efficient* de la familia **Gemini 2.0**, optimizado específicamente para tareas de alto volumen y alta frecuencia donde la latencia es un factor determinante para la experiencia del usuario. La capacidad de este modelo para generar salidas de alta calidad a una fracción del costo de sus predecesores permite que **Spin Compass** maneje un volumen creciente de consultas en **Slack** sin comprometer la viabilidad financiera del proyecto.

El benchmarking realizado indica que **Gemini 2.0 Flash-Lite** mantiene la robustez necesaria para procesos **RAG** complejos, ofreciendo una ventana de contexto de un millón de *tokens* que permite al sistema procesar hasta $1,500$ páginas de manuales y políticas de una sola vez. Esta capacidad es fundamental para Spin, donde la información se encuentra dispersa en documentos de extensión considerable como la Política de Viajes o los manuales de equipos de cómputo. Al utilizar **Gemini 2.0 Flash-Lite**, el sistema puede no solo recuperar fragmentos específicos, sino también comprender el contexto macro de la documentación, reduciendo la probabilidad de que se omita información crítica debido a limitaciones de memoria del modelo.

## 2.2. Rendimiento en Latencia y Ventana de Contexto

Uno de los requisitos no funcionales más estrictos para **Spin Compass** es la entrega de respuestas rápidas, idealmente en menos de $5$ segundos, para mantener la fluidez de las

conversaciones en **Slack**. Los resultados previos mostraron que **Gemini 2.0 Flash** lograba una latencia de $0.53$ segundos en la fase de generación. Si bien algunos experimentos sugieren que modelos «Lite» podrían presentar latencias ligeramente superiores en tareas de generación masiva debido a la priorización de recursos en la API de **Google**, para las tareas de síntesis y respuesta breve requeridas por el asistente de Spin, **Gemini 2.0 Flash-Lite** demuestra un rendimiento excepcional. En pruebas de orquestación de voz y texto, se han registrado tiempos de respuesta hacia el último token de $0.93$ segundos para bases de conocimiento de $50$ páginas, lo que sitúa al modelo Lite dentro de los umbrales de aceptación para Spin.

La ventana de contexto de un millón de *tokens* abre la posibilidad de implementar técnicas de **Model-Assisted Generation (MAG)** o **Long Context RAG**. Bajo este paradigma, en lugar de depender exclusivamente de la recuperación de fragmentos pequeños, el sistema puede cargar secciones enteras de documentos en el prompt del sistema, aprovechando la capacidad del modelo para encontrar la «aguja en el pajar» (**Needle In A Haystack**) con una precisión que a menudo supera al **RAG** tradicional en escenarios de búsqueda específica. Para **Spin Compass**, esto significa que ante consultas complejas que requieren relacionar múltiples políticas, **Gemini 2.0 Flash-Lite** puede actuar como un integrador de información superior, siempre que los datos que alimentan esa ventana estén libres de ruido estructural.

| **Componente de la Arquitectura** | **Configuración *Baseline* (Avance 3)** | **Configuración Alternativa (Avance 4)** |
|---|---|---|
| Modelo Generativo | Gemini 2.0 *Flash* | Gemini 2.0 *Flash-Lite* |
| Estrategia de *Chunking* | Max-Min *Semantic Chunking* | Max-Min con Limpieza Refinada |
| Base de Datos Vectorial | AlloyDB (*Dense Retrieval*) | AlloyDB con Optimización *HNSW* |
| Mecanismo de Limpieza | *NoiseReducer* Estándar | *NoiseReducer* Avanzado (*Regex GCP-Ready*) |
| Canal de Comunicación | *Slack API* | *Slack API* con Gestión de Hilos |

Tabla 1: Comparativa de configuraciones de arquitectura entre el estado base (Avance 3) y la propuesta optimizada (Avance 4).

# 3. Restricciones Técnicas en Google Cloud Platform y su Impacto en el RAG

## 3.1. Gestión de Infraestructura y Límites de Carga

La implementación de **Spin Compass** en **Google Cloud Platform** requiere una alineación estricta con las capacidades de servicios como **Cloud Run**, **AlloyDB** y **Vertex AI**. **Cloud Run**, utilizado para desplegar los endpoints conversacionales, impone límites en el tamaño de los payloads y en el tiempo de ejecución de las peticiones. Esto obliga a que la transferencia de texto desde la base de datos vectorial hacia el modelo de lenguaje sea extremadamente eficiente. El envío de texto redundante o ruidoso no solo aumenta la latencia de red, sino que también consume innecesariamente *tokens* de entrada en **Vertex AI**, afectando directamente el rendimiento del sistema bajo carga.

Adicionalmente, el **Files API de Google** es la vía recomendada para el manejo de documentos de gran tamaño, permitiendo desacoplar la carga del archivo de las solicitudes de generación de contenido. Esta restricción técnica implica que el proceso de limpieza debe ocurrir idealmente durante la fase de ingesta (Indexing), transformando el contenido bruto del PDF en un formato normalizado que pueda ser almacenado y recuperado sin las fricciones propias de los metadatos de archivos binarios. La eficiencia en el uso de *tokens* por minuto (TPM), que para proyectos estándar en ciertas regiones de GCP se sitúa en torno a los 3.4 millones, exige que **Spin Compass** no desperdicie capacidad en procesar avisos de derechos reservados.

## 3.2. Privacidad y Seguridad de la Información (PII)

Como sistema diseñado para el área de Finanzas y Recursos Humanos, **Spin Compass** debe manejar información sensible con los más altos estándares de privacidad. Las reglas de negocio prohíben explícitamente la exposición de datos personales o sensibles de los colaboradores. **GCP** ofrece herramientas como **Secret Manager** para la gestión de claves, pero la «limpieza» de los documentos debe ir más allá de lo estructural para incluir la anonimización de datos sensibles que pudieran estar presentes en manuales o ejemplos de políticas.

La modificación de los patrones de limpieza debe integrar filtros proactivos que identifiquen y eliminen patrones de correos electrónicos, números telefónicos o identificadores internos que no sean necesarios para la comprensión de la política general. Esta limpieza preventiva en la fase de **RAG** reduce el riesgo de que el modelo **Gemini 2.0 Flash-Lite**, a pesar de sus capas de seguridad nativas, pueda «filtrar» información contextual sensible en sus respuestas. La coherencia con las políticas internas de Spin y el respeto total a la privacidad son pilares que el **NoiseReducer** debe defender mediante expresiones regulares validadas específicamente para el formato de datos de la organización.

# 4. Rediseño de Patrones de Limpieza de Documentos (NoiseReducer 2.0)

### Optimización de Expresiones Regulares para Ruido Estructural

La clase **NoiseReducer** identificada en el *baseline* operaba bajo principios de limpieza de texto plano, centrada en minúsculas y eliminación de espacios. Sin embargo, para satisfacer las demandas de **Gemini 2.0 Flash-Lite** y las restricciones de **GCP**, se requiere una evolución hacia patrones de limpieza basados en expresiones regulares (**Regex**) de alta precisión. Los documentos PDF, fuente principal de conocimiento de Spin, generan ruido al ser convertidos a texto, incluyendo saltos de línea huérfanos, encabezados repetitivos en cada página y pies de página con información legal irrelevante.

La eliminación de estos elementos no es meramente estética; su presencia distorsiona el cálculo de similitud de coseno en la búsqueda vectorial. Por ejemplo, si un encabezado dice «Confidencial - Propiedad de Spin», este patrón se repite en todas las páginas, lo que puede causar que el motor de búsqueda recupere fragmentos de páginas aleatorias simplemente porque comparten esa cadena de texto común, diluyendo la relevancia del contenido normativo real. La nueva estrategia de limpieza debe implementar patrones para detectar y remover estas estructuras antes del proceso de *embedding*.

| Tipo de Ruido | Patrón de Identificación (*Regex*) | Justificación Semántica |
|---|---|---|
| Números de Página | `^Página \d+ de \d+$` | Evita que el modelo cite el número de página como parte de un valor financiero. |
| Encabezados de Marca | `^Spin Compass.*Política de Viajes$` | Remueve redundancia que distorsiona la densidad de palabras clave. |
| Avisos Legales | `^©.*Todos los derechos reservados.*$` | Libera espacio en la ventana de contexto y reduce el conteo de *tokens*. |
| Metadatos Técnicos | `\{S\d+\} \/ \{E\d+\}` | Elimina etiquetas de sistema que interfieren con la tokenización del *LLM*. |
| Enlaces Web | `http[s]?://(?:[a-zA-Z0-9]\|[$-_@.&+]\|[!*(),]\|(?:%[0-9a-fA-F][0-9a-fA-F]))+` | Limpia referencias externas que no aportan valor al contexto de búsqueda. |

Tabla 2: Definición de patrones de limpieza mediante expresiones regulares y su impacto en la calidad semántica del contexto.

## 4.1. Tratamiento de Tablas y Datos Estructurados en PDFs

Un desafío recurrente identificado en el Avance 3 es el «corte» de tablas de viáticos o restricciones, lo que provoca que la característica de integridad se degrade. Si el sistema de limpieza y segmentación no reconoce la estructura de una tabla, puede fragmentar una regla de negocio a la mitad (ej. separar el destino del monto permitido). La investigación sugiere que el uso de formatos intermedios como **Markdown** es altamente efectivo para preservar la relación entre encabezados de columna y valores en sistemas **RAG**.

En lugar de tratar las tablas como texto plano indiscriminado, el **NoiseReducer** debe ser capaz de identificar estructuras tabulares y, mediante patrones de limpieza específicos, asegurar que los delimitadores (como pipes | o tabs) se mantengan de forma coherente. Esto permite que el método **Max-Min Semantic Chunking** opere sobre bloques de datos que el modelo **Gemini 2.0 Flash-Lite** puede interpretar correctamente como estructuras relacionales. La eficacia del RCC (Relevant Chunk Contained) subió al 73% con segmentación semántica, y se proyecta que con una limpieza orientada a la estructura de tablas, este valor supere el 80%.

## 4.2. Gestión de Stopwords y Normalización de Dominio

La eliminación de palabras vacías o *stopwords* debe ser refinada para el dominio de Spin. Mientras que bibliotecas como **NLTK** proporcionan listas genéricas (artículos, preposiciones), el sistema de **Spin Compass** requiere un enfoque híbrido. Existen términos técnicos o corporativos que podrían actuar como «ruido» en ciertos contextos pero ser vitales en otros. El **NoiseReducer** debe integrar un diccionario de términos excluidos que sea dinámico, permitiendo que palabras como «Spin», «Spinner» o «Finanzas» no dominen el vector semántico a menos que la consulta del usuario así lo exija.

La normalización también incluye la corrección de errores comunes de OCR y la estandarización de formatos de fecha y moneda. En el contexto de gastos de viaje, es crítico que «100 MXN», «100 pesos» y «cien pesos» sean tratados bajo una lógica de limpieza que facilite su recuperación. La consistencia en el uso de comillas, apóstrofes y caracteres especiales no alfanuméricos asegura que el modelo Lite reciba un contexto limpio, minimizando el esfuerzo cognitivo del modelo para «descifrar» texto mal formateado y permitiéndole enfocarse en la síntesis de la respuesta.

# 5. Análisis de Impacto en las Métricas de Desempeño

## 5.1. Mejora Proyectada en el Hit Rate y MRR

El **Hit Rate (HR)** es la métrica que determina si el sistema es capaz de localizar el documento correcto ante una consulta. En el *baseline*, el **HR@5** alcanzó un impresionante $93.3\%$ utilizando **Max-Min Semantic Chunking**. No obstante, este resultado se obtuvo en un entorno controlado con documentos relativamente limpios. Al escalar a la producción real en **GCP** con documentos de diversas fuentes, el riesgo de subajuste (fallo de recuperación) aumenta debido a la mayor variabilidad del ruido. La implementación de los nuevos patrones de limpieza tiene como objetivo estabilizar y potencialmente elevar esta métrica al eliminar las «distracciones» semánticas que causan que el buscador vectorial se desvíe hacia fragmentos irrelevantes pero estructuralmente similares.

Por otro lado, el **Mean Reciprocal Rank (MRR)** mide qué tan cerca de la primera posición aparece el fragmento relevante. Un $\mathbf{MRR}$ de $0.78$ indica que la respuesta correcta suele estar en la primera o segunda posición. Al limpiar los metadatos y encabezados redundantes, se reduce la densidad de información no pertinente en el espacio vectorial, lo que permite que el fragmento de «verdad» (Gold Chunk) destaque con mayor claridad frente a la consulta del usuario. Un incremento en el $\mathbf{MRR}$ es fundamental para **Gemini 2.0 Flash-Lite**, ya que este modelo se beneficia de tener la información más relevante al inicio del contexto recuperado para evitar el fenómeno de «pérdida en el medio» (**lost-in-the--middle**) común en modelos de lenguaje.

## 5.2. Evaluación de la Fidelidad (Faithfulness) y RCC

La fidelidad de la respuesta generada es un indicador de que el modelo se adhiere exclusivamente al contexto recuperado y no alucina con conocimientos generales. Para **Spin Compass**, cualquier alucinación en temas de políticas de viáticos es considerada un fallo

crítico por el área de Finanzas. La limpieza profunda de los documentos reduce la probabilidad de alucinaciones al eliminar información contradictoria u obsoleta que pudiera haber quedado en fragmentos de versiones anteriores de una política.

El **Relevant Chunk Contained (RCC)** mide si el fragmento recuperado contiene la unidad de información completa. La modificación de los patrones de limpieza influye en el **RCC** al asegurar que los límites de los fragmentos no sean definidos por elementos de ruido. Si un *chunk* se corta justo donde empieza un encabezado de página, la regla de negocio queda incompleta. Al pre-procesar el documento y remover estos elementos, el algoritmo de segmentación semántica puede unir oraciones que antes estaban separadas por una ruptura de página física, mejorando la cohesión del bloque informativo y, por ende, el **RCC** final.

| **Métrica de Desempeño** | ***Baseline* (Avance 3)** | **Objetivo (Avance 4)** | **Mecanismo de Mejora** |
|---|---|---|---|
| *Hit Rate @5* | 93.3% | $> 95\%$ | Eliminación de ruido semántico coincidente. |
| *MRR* | 0.78 | $> 0.82$ | Mayor pureza del vector de consulta vs. documento. |
| *RCC* | 0.73 | $> 0.80$ | Unión de oraciones fragmentadas por ruido de página. |
| *Faithfulness (Score)* | Alto | Máximo | Reducción de contexto ambiguo o contradictorio. |
| *Latencia Promedio* (s) | 0.53 | $< 0.50$ | Procesamiento de fragmentos más concisos y limpios. |

Tabla 3: Comparación de métricas de desempeño entre la línea base y los objetivos del próximo avance, detallando los mecanismos de mejora asociados.

# 6. Integridad de la Información y Gestión de Alucinaciones

## 6.1. La «Verdad Documental» en Finanzas

El principal riesgo identificado para **Spin Compass** no es la elocuencia de la respuesta, sino la recuperación de una política obsoleta. En el área de Finanzas, donde los presupuestos y las reglas de reembolso cambian anualmente, la integridad de la base de conocimientos es suprema. El sistema debe demostrar que es capaz de recuperar la versión vigente de la política, descartando cualquier fragmento que, por falta de limpieza, pudiera contener fechas o montos antiguos. El uso de **Gemini 2.0 Flash-Lite**, con su capacidad de razonamiento mejorada sobre versiones anteriores, permite detectar estas inconsistencias si el proceso de limpieza ha etiquetado o filtrado correctamente los metadatos de vigencia.

Las alucinaciones suelen ocurrir cuando el modelo intenta «rellenar huecos» en la información proporcionada. Si el proceso de limpieza es demasiado agresivo y elimina datos cruciales, o si es demasiado laxo y permite ruido contradictorio, el modelo se ve forzado a inferir políticas, lo cual viola una de las reglas de negocio fundamentales de Spin: «Si no existe un documento válido, debe decirlo». Por tanto, la limpieza debe ser equilibrada (*iterative approach*), comenzando con lo estructural y moviéndose hacia lo semántico solo bajo validación empírica.

## 6.2. Model-Assisted Generation (MAG) vs. RAG Tradicional

La investigación reciente sobre **Gemini 2.0 Flash-Lite** sugiere que para bases de conocimiento de tamaño moderado (como las de Spin en su fase inicial), el enfoque **MAG** podría ofrecer una precisión cercana al 100%. Al colocar toda la documentación limpia en el prompt del sistema, se elimina el riesgo de fallo en la recuperación vectorial. Sin embargo,

esta estrategia tiene un costo de *tokens* significativamente mayor y puede aumentar la latencia a medida que la organización crece y se incorporan más categorías como onboarding, beneficios y políticas técnicas.

Para **Spin Compass**, la estrategia recomendada es mantener el **RAG** como arquitectura base para garantizar la trazabilidad mediante citas a la fuente original, pero utilizar la potencia de **Gemini 2.0 Flash-Lite** para realizar un paso de «verificación cruzada» cuando la consulta sea ambigua. Esta hibridación permite lo mejor de ambos mundos: la eficiencia y escalabilidad del **RAG** con la solidez interpretativa del modelo Lite. La limpieza de los documentos es, de nuevo, el factor habilitador, ya que fragmentos limpios facilitan tanto la búsqueda vectorial como la síntesis del modelo.

# 7. Consideraciones Operativas y Experiencia del Spinner

## 7.1. Integración en Slack y Tono de Comunicación

**Spin Compass** no solo debe ser preciso, sino también accesible. Su integración en **Slack**, el canal de comunicación central de Spin, exige que las respuestas sean claras, accionables y mantengan el tono cálido y humano que define la cultura de la organización. La limpieza de los documentos influye indirectamente en este tono: si el modelo recibe texto plagado de jerga legal innecesaria o errores de OCR, la respuesta tiende a volverse técnica o confusa. Un contexto limpio permite que el modelo **Gemini 2.0 Flash-Lite** se concentre en «traducir» la política institucional a un lenguaje amigable para el Spinner final.

El manejo de hilos (threads) en **Slack** requiere que el sistema mantenga una memoria a corto plazo del contexto de la conversación. Las restricciones de **GCP** en cuanto a latencia y gestión de estados obligan a que este contexto también sea procesado por el **NoiseReducer** para evitar que el historial de la charla se vuelva una fuente de ruido en sí misma. La capacidad del modelo Lite para manejar flujos guiados y escalamiento automático a soporte

humano asegura que el Spinner nunca se sienta desatendido, incluso cuando la política solicitada no se encuentre en la base de conocimientos.

## 7.2. Costos y Sostenibilidad a Largo Plazo

El análisis financiero de la solución muestra que **AlloyDB** representa el gasto principal (USD 470 de los 600 mensuales estimados). Dado que este costo es relativamente fijo, la optimización debe centrarse en el consumo de *tokens* de **Gemini**, donde **Gemini 2.0 Flash-Lite** ofrece ahorros significativos. La reducción del ruido documental impacta directamente en este renglón: menos *tokens* de ruido equivalen a un menor costo por consulta y a la posibilidad de atender a más Spinners con el mismo presupuesto.

| **Componente de Costo (GCP)** | **Monto Mensual (USD)** | **Estrategia de Optimización** |
|---|---|---|
| AlloyDB (Base de Conocimiento) | $470 | Optimización de índices y almacenamiento de *embeddings*. |
| Gemini 2.0 *Flash-Lite* | $80 | Limpieza de ruido para reducir consumo de *tokens* por *prompt*. |
| Cloud Run | $35 | Reducción de tamaño de *payloads* para bajar latencia y ancho de banda. |
| Otros (*Artifact Registry*, etc.) | $15 | Gestión eficiente de despliegues y secretos. |
| **Total Estimado** | **$600** | **Costo anual de approx. USD $7,800** |

Tabla 4: Estimación mensual de costos en *Google Cloud Platform* y estrategias de optimización financiera.

# 8. Resultados

## 8.1. Entorno Experimental y Descripción del Notebook de Calibración

En notebook constituye el núcleo técnico de validación y calibración del sistema **Spin Compass**. Este artefacto no se limita a implementar el pipeline **Retrieval-Augmented Generation (RAG)**, sino que funciona como plataforma controlada de experimentación para medir, comparar y justificar empíricamente decisiones arquitectónicas adoptadas en el Avance 4.

Su propósito principal es transformar hipótesis conceptuales —como el impacto del ruido estructural en el espacio vectorial o la eficiencia operativa de **Gemini 2.0 Flash-Lite**— en resultados cuantificables mediante métricas reproducibles.

### 8.1.1. Arquitectura General del Notebook

El notebook está estructurado en módulos que reflejan el flujo operativo del sistema en producción:

1. **Inicialización del entorno y configuración global.**
2. **Ingesta y preprocesamiento documental (Limpieza de documentos).**
3. **Implementación del Algoritmo Max-Min Semantic Chunking.**
4. **Implementación del pipeline RAG y Evaluación cuantitativa de desempeño.**
5. **Configuración del Cliente Gemini.**
6. **Generacion del Golden Data set.**
7. **Motor de evaluacion y Reporte de Metricas de RAG.**

Esta organización modular permite ejecutar experimentos de manera incremental, garantizando trazabilidad entre modificaciones introducidas y variaciones observadas en las métricas de desempeño.

### 8.1.2. Inicialización del entorno y configuración global

En esta fase de inicialización del entorno experimental del sistema. Su objetivo es preparar todos los componentes necesarios para ejecutar posteriormente el pipeline de recuperación semántica y generación de respuestas. En esta etapa no se realiza todavía ningún proceso de búsqueda o generación, sino que se configuran las dependencias, rutas de almacenamiento y modelos fundamentales que permitirán el funcionamiento del sistema **RAG**.

La sección también establece constantes relacionadas con los modelos utilizados. Se define el tokenizador "cl100k_base", que es compatible con modelos de la familia **GPT-4** y permite calcular con precisión el número de *tokens* que consumirán las consultas y respuestas. Esto es fundamental para estimar costos y controlar el tamaño de los prompts. Asimismo, se especifica el modelo de embeddings "paraphrase-multilingual-MiniLM-L12-v2", un modelo multilingüe liviano diseñado para generar representaciones vectoriales densas de textos en distintos idiomas.

Finalmente, se inicializan los modelos en memoria. Se carga el tokenizador para habilitar el conteo de *tokens* y se instancia el modelo de embeddings mediante **SentenceTransformer**. Este último será responsable de convertir tanto los fragmentos documentales como las consultas del usuario en vectores numéricos dentro de un espacio semántico compartido, lo cual es esencial para realizar búsqueda por similitud coseno en etapas posteriores.

### 8.1.3. Ingesta y Preprocesamiento Documental

Primero, se define una función para extraer texto de archivos PDF utilizando **PyMuPDF**, leyendo el contenido página por página y concatenándolo en un solo bloque de texto. Luego, se implementa una función que recorre recursivamente un directorio raíz y carga documentos en formato PDF, TXT o MD, almacenándolos en un **DataFrame** con su nombre, ruta y contenido textual.

El componente central es la clase **NoiseReducer**, diseñada para eliminar ruido estructural frecuente en documentos corporativos, como numeración de páginas, encabezados repetitivos, avisos legales, metadatos administrativos y campos de firma. Se definen dos

conjuntos de patrones basados en expresiones regulares: uno básico y otro más exhaustivo, permitiendo alternar entre distintos niveles de limpieza para experimentación. El método de limpieza aplica estas reglas y normaliza el espaciado del texto.

Posteriormente, se incluye una función de normalización que corrige saltos de línea incorrectos derivados de la extracción de PDF, uniendo líneas fragmentadas en párrafos coherentes. Finalmente, el texto se divide en oraciones utilizando puntuación terminal básica, preparando el contenido para el proceso de **chunking** semántico.

En conjunto, esta sección prepara el corpus documental eliminando ruido estructural y organizando el texto en unidades manejables, lo que mejora la calidad de la representación vectorial y la precisión del sistema **RAG** en etapas posteriores.

### 8.1.4. Implementación del Algoritmo Max-Min Semantic Chunking

El objetivo de esta sección es segmentar el texto en fragmentos semánticamente coherentes en lugar de usar divisiones por tamaño fijo. Para ello, el texto se divide primero en oraciones y cada una se convierte en un **embedding** vectorial.

El algoritmo construye los **chunks** de forma progresiva. Para cada nueva oración, calcula su similitud con las oraciones ya incluidas en el fragmento actual. Si la similitud supera un umbral mínimo, la oración se agrega al mismo **chunk**; de lo contrario, se cierra el fragmento y se inicia uno nuevo.

Cuando el **chunk** contiene varias oraciones, el criterio se vuelve más estricto mediante un umbral dinámico que depende tanto de la cohesión interna del fragmento como de su tamaño. Esto evita que fragmentos largos incorporen contenido débilmente relacionado.

En resumen, este método garantiza alta coherencia semántica dentro de cada fragmento, lo que mejora la calidad de recuperación en sistemas **RAG** y reduce la fragmentación de información relevante.

**Estadísticas descriptivas de longitud de fragmentos para la Configuración P1.**

| count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|
| 417.0 | 84.235012 | 101.273917 | 2.0 | 34.0 | 58.0 | 106.0 | 1301.0 |

Los resultados muestran la generación de 417 fragmentos con una media de 84.23 tokens y una desviación estándar de 101.27. La amplitud entre el valor mínimo (2 tokens) y máximo (1301 tokens) evidencia alta variabilidad, atribuible a la naturaleza heterogénea del contenido documental.

**Estadísticas descriptivas de longitud de fragmentos para la Configuración P2.**

| count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|
| 378.0 | 92.060847 | 110.865986 | 3.0 | 30.25 | 69.0 | 112.0 | 1299.0 |

En la configuración P2 se observa una reducción del número total de fragmentos (378) junto con un incremento en la media de longitud (92.06 tokens). Esto sugiere una mayor consolidación semántica de los chunks, con menor fragmentación y potencial mejora en la coherencia contextual para la recuperación en el sistema **RAG**.

### 8.1.5. Implementación del Pipeline RAG y Evaluación del Desempeño

En esta sección se implementa el pipeline principal de ejecución del sistema, orquestando todas las etapas desde la carga de documentos hasta la generación de estadísticas finales del proceso de **chunking**.

En primer lugar, el sistema carga los documentos ubicados en el directorio configurado. Si no se encuentran archivos válidos (PDF, TXT o MD), la ejecución se detiene y se notifica al usuario. Una vez cargados los documentos, se procede a la limpieza y normalización del texto utilizando dos configuraciones distintas del **NoiseReducer**: una limpieza básica (Patrón 1) y una limpieza avanzada (Patrón 2). Esto permite comparar experimentalmente el impacto de diferentes niveles de eliminación de ruido estructural.

Posteriormente, se aplica el algoritmo **Max-Min Semantic Chunking** sobre ambos textos limpios, generando dos conjuntos de fragmentos semánticos por documento (P1 y P2). Cada conjunto representa una estrategia distinta de preprocesamiento.

En la etapa final, el sistema calcula estadísticas sobre los fragmentos generados, particularmente la distribución de tokens por **chunk** utilizando el tokenizador configurado. Se generan métricas descriptivas —media, mínimo, máximo y desviación estándar— que permiten evaluar el tamaño y la variabilidad de los fragmentos producidos.

Finalmente, los resultados se exportan a archivos CSV independientes para cada configuración, facilitando su análisis posterior y comparación cuantitativa.

En síntesis, esta sección integra todas las fases anteriores en un flujo completo y reproducible, permitiendo evaluar cómo distintas estrategias de limpieza afectan la segmentación semántica y la distribución de tokens del corpus resultante.

Aquí también se realizan experimentos comparativos entre la arquitectura **baseline** y la versión optimizada. Cada configuración es evaluada utilizando el mismo conjunto de consultas de prueba.

Primero, se define `doc_idx = 0`, lo que indica que se trabajará con el primer documento cargado en el **DataFrame** `df_docs`. Luego, se imprime una vista preliminar (los primeros 500 caracteres) del texto limpio utilizando el Patrón 1 (`clean_text_p1`). Posteriormente, se hace lo mismo con el texto limpio utilizando el Patrón 2 (`clean_text_p2`).

El objetivo de este bloque es inspeccionar visualmente las diferencias entre la limpieza básica (10 patrones) y la limpieza avanzada (55 patrones). Esto permite verificar qué elementos de ruido estructural fueron eliminados adicionalmente por el segundo conjunto de expresiones regulares, como encabezados administrativos, metadatos, cláusulas legales repetitivas o campos de firma.

En síntesis, este fragmento no modifica datos ni ejecuta métricas; únicamente permite realizar una validación cualitativa manual del impacto de cada estrategia de limpieza sobre el contenido textual antes del proceso de **chunking**.

### 8.1.6. Generación del Golden Dataset con LLM (Gemini)

En esta sección se implementa la generación automática de un **Golden Dataset** utilizando un modelo de lenguaje (**Gemini 2.0 Flash**). El objetivo de esta etapa es crear un conjunto de preguntas de evaluación cuya respuesta esté explícitamente contenida en fragmentos específicos del corpus, permitiendo medir posteriormente el desempeño del sistema **RAG** en tareas de recuperación.

Primero, se define una función que recibe un fragmento de texto (**chunk**) y genera, mediante **Gemini**, una pregunta compleja y específica basada únicamente en ese contenido. El **prompt** está cuidadosamente diseñado para asegurar que: (1) la pregunta sea autónoma, (2) no haga referencia explícita al texto fuente y (3) su respuesta pueda encontrarse directamente en el fragmento proporcionado. Se incluye manejo de errores y una pausa controlada entre llamadas para respetar límites de la API.

Posteriormente, la función `create_golden_dataset_llm` construye el dataset completo. Para ello, filtra los fragmentos que se encuentran dentro de un rango válido de tokens (entre 50 y 800), evitando textos demasiado cortos o excesivamente largos. Luego, selecciona aleatoriamente una muestra configurable y genera una pregunta por cada fragmento elegido. Cada entrada del **Golden Dataset** incluye un identificador único, la pregunta generada, el identificador del fragmento original, el texto base (**ground truth**) y el archivo fuente.

Finalmente, el sistema ejecuta este proceso para ambas configuraciones de limpieza (P1 y P2), generando dos **Golden Datasets** independientes. Los resultados se exportan a archivos CSV y se almacenan en memoria para evaluaciones posteriores.

En síntesis, esta sección transforma el corpus segmentado en un conjunto de pruebas controladas que permite evaluar cuantitativamente la capacidad del sistema **RAG** para recuperar el fragmento correcto ante consultas formuladas de manera realista.

### 8.1.7. Motor de Evaluación y Reporte de Métricas RAG

La Sección 7 implementa el motor de evaluación del sistema **RAG**, encargado de medir cuantitativamente la calidad de recuperación utilizando el **Golden Dataset** previamente generado.

Primero, se define la función `retrieve_top_k`, que recibe una pregunta en lenguaje natural, codifica la consulta en el mismo espacio de **embeddings** del corpus y recupera los `k` fragmentos más similares mediante similitud coseno. Esta función devuelve los identificadores de los **chunks** recuperados, su puntaje de similitud, el texto correspondiente y el archivo fuente.

Posteriormente, la función `calculate_metrics` evalúa el desempeño del sistema comparando los resultados recuperados contra el **Golden Dataset**. Para cada consulta, se calculan cuatro métricas principales:

**Hit Rate:** proporción de consultas en las que el fragmento exacto de referencia aparece dentro de los Top-k resultados.

**MRR (Mean Reciprocal Rank):** mide qué tan arriba aparece el primer resultado correcto, penalizando posiciones más bajas.

**SRA (Same Relevant Article):** verifica si el primer resultado pertenece al mismo documento fuente que el fragmento correcto.

**RCC (Relevant Chunk Contained):** evalúa si el fragmento correcto está contenido dentro del primer resultado recuperado, incluso si no es exactamente el mismo **chunk**.

Para cada método de **chunking** (Max-Min P1 y Max-Min P2), el sistema precalcula los **embeddings** del corpus —una operación computacionalmente costosa— y luego evalúa todas las consultas del **Golden Dataset**. Las métricas se acumulan y finalmente se promedian sobre el total de preguntas.

Al finalizar, se genera un reporte comparativo que muestra el desempeño de cada configuración y selecciona automáticamente el mejor método según la métrica RCC, considerada clave para garantizar integridad semántica en sistemas corporativos.

**Reporte de Métricas – Spin Compass RAG**

**Comparación de métricas de desempeño para configuraciones Max-Min P1 y P2.**

| | Hit Rate | MRR | SRA | RCC |
|---|---|---|---|---|
| **Max-Min P1** | 0.866667 | 0.733333 | 0.733333 | 0.600000 |
| **Max-Min P2** | 0.533333 | 0.402222 | 0.933333 | 0.333333 |

**Mejor método según RCC: Max-Min P1**

Hit Rate : 86.67%
MRR : 0.7333
SRA : 73.33%
RCC : 60.00%

En síntesis, esta sección convierte el pipeline experimental en un marco de evaluación formal, permitiendo comparar objetivamente distintas estrategias de limpieza y segmentación dentro del sistema **RAG**. Los resultados evidencian que la configuración Max-Min P1 ofrece un balance superior entre recuperación exacta y coherencia semántica contenida, consolidándose como la estrategia recomendada para la Fase 1 del sistema **Spin Compass**.

# 9. Conclusiones y Recomendaciones para la Implementación

La transición hacia modelos alternativos en el Avance 4 de **Spin Compass** consolida la visión de un asistente robusto, eficiente y alineado con los estándares técnicos de **Google Cloud Platform**. La adopción de **Gemini 2.0 Flash-Lite** no es solo un cambio de versión, sino una decisión técnica fundamentada en la necesidad de manejar grandes volúmenes de contexto con una latencia mínima y costos controlados. Los resultados del *baseline* han sido

superados conceptualmente al integrar una estrategia de limpieza de documentos mucho más agresiva y específica, diseñada para mitigar los fallos de recuperación que antes se aceptaban como ruido inevitable.

**Refinamiento Quirúrgico del NoiseReducer:** Es imperativo implementar los nuevos patrones de expresiones regulares identificados para remover de forma definitiva encabezados, pies de página y metadatos técnicos. Esta acción es el habilitador principal para elevar el **RCC** por encima del $80\%$ y reducir el costo de *tokens* en **Vertex AI**.

**Validación de Gemini 2.0 Flash-Lite:** El modelo demuestra ser el sucesor ideal para **Gemini 2.0 Flash**, ofreciendo una ventana de contexto de $1M$ de *tokens* y una latencia competitiva. Se recomienda su uso en todas las fases de PoC y Fase 1 para validar su precisión en escenarios reales de **Slack**.

**Hibridación RAG-MAG:** Se debe explorar el uso de la ventana de contexto extendida para realizar síntesis de alto nivel en consultas complejas, mientras se mantiene el **RAG** tradicional para consultas de hechos simples y para garantizar la citación de fuentes.

**Monitoreo y Mejora Continua:** La implementación de métricas de observabilidad en **GCP** debe incluir no solo el **Hit Rate** y el **MRR**, sino también el conteo de *tokens* efectivos vs. *tokens* de ruido. Esto permitirá realizar ajustes finos a los patrones de limpieza a medida que se incorporen nuevas categorías de documentos en las fases siguientes del proyecto.

En conclusión, el enfoque de **Spin Compass** se fortalece al reconocer que la inteligencia del modelo de lenguaje es tan buena como la calidad de los datos que consume. Al optimizar la limpieza de documentos bajo las restricciones de **GCP**, Spin no solo construye un bot más inteligente, sino una infraestructura de conocimiento resiliente, escalable y profundamente respetuosa de la integridad de sus procesos financieros y humanos.

# 10. Referencias

[1] L. Visengeriyeva, A. Kammer, I. Bär, A. Kniesz, y M. Plöd, «CRISP-ML(Q): The ML lifecycle process». Accedido: 22 de enero de 2026. [En línea]. Disponible en: https://ml-ops.org/content/crisp-ml

[2] R. Hernández-Sampieri y C. Mendoza, *Metodología de la investigación: Las rutas de la investigación cuantitativa, cualitativa y mixta*, 3.ª ed. McGraw-Hill, 2023.

[3] A. Moreno-Cediel, E. Garcia-Lopez, D. De-Fitero-Dominguez, y A. Garcia-Cabot, «Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries». Alcalá de Henares, Madrid, Spain, 2025.

[4] C. Kiss, M. Nagy, y P. Szilagyi, «Max–Min semantic chunking of documents for RAG application». Budapest, Hungary, 2025.